

1 Точка входа

Открываем `src/index.tsx`



React берет root из HTML.

В файле `index.html` есть элемент с `id="root"`.



`createRoot(...).render(<App />)`

отрисовывает приложение.

React «монтирует» наш компонент в DOM.



`App` — главный компонент.

С него начинается дерево компонентов приложения.

```
src/index.tsx

1  import React from 'react';
2  import { createRoot } from 'react-dom/client';
3  import App from './App';
4
5  const rootElement = document.getElementById('root');
6  const root = createRoot(rootElement!);
7
8  root.render(<App />);
```

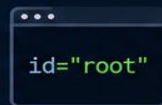


Итог

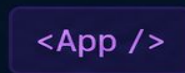
Этот файл — точка входа в React-приложение. Отсюда начинается вся отрисовка.



index.html



React



главный компонент

2 Mock-данные

Открываем `src/mocks/products.ts`



Пока нет сервера

данные можно хранить в mock-файле.



Product — тип одного товара.

Определяет, какие поля есть у каждого товара.



products — массив товаров.

Содержит список всех товаров для отображения.



id понадобится для key

в списках компонентов.
Он должен быть уникальным.



Важно

Mock-данные нужны только для разработки.
Позже их заменит реальный API.

TS

src/mocks/products.ts

```
1  export type Product = {
2    id: number;
3    title: string;
4    category: string;
5    price: number;
6    inStock: boolean;
7  };
8
9  export const products: Product[] = [
10    {
11      id: 1,
12      title: "Apple iPhone 15",
13      category: "Smartphones",
14      price: 999,
15      inStock: true,
16    },
17    {
18      id: 2,
19      title: "Sony WH-1000XM5",
20      category: "Headphones",
21      price: 349,
22      inStock: true,
23    },
24    // ... другие товары
25  ];
```

3

Компонент карточки

Открываем `src/components/ProductCard.tsx`



Props — данные, которые компонент получает снаружи. `ProductCard` получает один товар.

Тип пропсов

```
type ProductCardProps = {  
  product: Product;  
};
```

Что делает компонент

- Показывает название товара (title).
- Показывает категорию (category).
- Показывает цену (price).
- Показывает наличие (inStock).



Важно

Компонент отвечает только за отображение одного товара.



ProductCard.tsx

```
1  import type { Product } from '../mocks/products';  
2  
3  type ProductCardProps = {  
4    product: Product;  
5  };  
6  
7  const ProductCard = ({ product }: ProductCardProps) => {  
8    return (  
9      <article className="product-card">  
10        <h2>{product.title}</h2>  
11        <p>Category: {product.category}</p>  
12        <p>Price: ${product.price}</p>  
13        <p>  
14          {product.inStock ? 'In stock' : 'Out of stock'}  
15        </p>  
16      </article>  
17    );  
18  };  
19  
20  export default ProductCard;
```

4

Компонент списка

Открываем `src/components/ProductList.tsx`



ProductList получает массив товаров через **props**.



Через **map** превращаем массив данных в массив компонентов. Каждый товар отображается как отдельная карточка.



Для каждого элемента нужен **key**. Это помогает React правильно обновлять список.

Основной фрагмент

```
{products.map((product) => (  
  <ProductCard key={product.id} product={product} />  
))}
```



Важно

key должен быть уникальным и стабильным, поэтому используем уникальный **id** товара.



ProductList.tsx

```
1  import type { Product } from '../mocks/products';  
2  import ProductCard from '../ProductCard';  
3  
4  type ProductListProps = {  
5    products: Product[];  
6  };  
7  
8  const ProductList = ({ products }: ProductListProps) => {  
9    return (  
10      <section className="product-list">  
11        <h2>Products</h2>  
12        {products.map((product) => (  
13          <ProductCard key={product.id} product={product} />  
14        ))}  
15      </section>  
16    );  
17  };  
18  
18  export default ProductList;
```


5

Пустой список

Показываем **отдельное состояние**,
когда товаров нет.



Зачем это нужно?

Если список пуст, пользователю не показываются
пустые карточки или пустая страница.
Мы явно сообщаем, что товаров нет.



Код в ProductList.tsx

```
if (products.length === 0) {  
  return <p className="empty">No products found</p>;  
}
```

Если массив `products` пустой (длина равна 0) —
возвращаем сообщение вместо списка.



Результат для пользователя

Есть товары



Apple iPhone 15
Smartphones • \$999



Sony WH-1000XM5
Headphones • \$349

Товаров нет



No products found



ProductList.tsx

```
1 import type { Product } from '../mocks/products';  
2 import ProductCard from './ProductCard';  
3  
4 type ProductListProps = {  
5   products: Product[];  
6 };  
7  
8 const ProductList = ({ products }: ProductListProps) => {  
9   if (products.length === 0) {  
10     return <p className="empty">No products found</p>;  
11   }  
12  
13   return (  
14     <section className="product-list">  
15       <h2>Products</h2>  
16       {products.map((product) => (  
17         <ProductCard key={product.id} product={product} />  
18       ))}  
19     </section>  
20   );  
21 };  
22  
23 export default ProductList;
```



Совет

Обработка пустых данных — важная часть UX.
Это делает приложение понятным и дружелюбным.

6

Кнопка и состояние

Открываем `src/App.tsx`



useState — это React-хук.

Он позволяет хранить состояние в функциональном компоненте.



Наше состояние

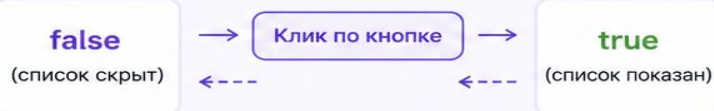
отвечает за видимость списка товаров (показывать или скрывать).

```
const [isProductListVisible, setIsProductListVisible] = useState(false);
```

`isProductListVisible` — текущее значение (булево).

`setIsProductListVisible` — функция для его обновления.

Как работает состояние



→ изменение состояния

---> обратное изменение состояния



tsx App.tsx

```

1  import { useState } from 'react';
2  import ProductList from '../components/ProductList';
3  import { products } from '../mocks/products';
4
5  const App = () => {
6    const [isProductListVisible, setIsProductListVisible] =
      useState(false);
7
8    const handleToggleProductList = () => {
9      setIsProductListVisible((currentValue) => !currentValue);
10   };
11
12   return (
13     <div className="app">
14       {/* остальной JSX позже */}
15     </div>
16   );
17   };
  
```



Что делает обработчик

```
setIsProductListVisible((currentValue) => !currentValue);
```

Берёт текущее значение состояния (`currentValue`)

и устанавливает противоположное: `true` → `false` или `false` → `true`.

7

Условная отрисовка

Показываем список только когда нужно.



Кнопка управляет видимостью списка

Текст кнопки меняется в зависимости от состояния.

Список скрыт (*false*)

Список показан (*true*)

Show products



Hide products



Условная отрисовка списка

Если *isProductListVisible* равно *true* — компонент *ProductList* отображается.
Если *false* — он не рендерится.

Как это работает



Важно

React перерисовывает интерфейс при изменении состояния. Это и есть реактивность.



tsx

App.tsx

```

1  import { useState } from 'react';
2  import ProductList from '../components/ProductList';
3  import { products } from '../mocks/products';
4
5  const App = () => {
6    const [isProductListVisible, setIsProductListVisible] =
7      useState(false);
8
9    const handleToggleProductList = () => {
10     setIsProductListVisible((currentValue) => !currentValue);
11   };
12
13   return (
14     <div className="app">
15       <h1>Product App</h1>
16
17       <button type="button" onClick={handleToggleProductList}>
18         {isProductListVisible ? 'Hide products' : 'Show products'}
19       </button>
20
21       {isProductListVisible && <ProductList products={products} />}
22     </div>
23   );
  
```

Текст кнопки зависит от состояния

Список отображается только если *isProductListVisible* равно *true*



Итог

- ✓ *useState* хранит состояние видимости списка.
- ✓ Кнопка меняет состояние.
- ✓ Оператор *&&* используется для условной отрисовки.

8

Практика

Попробуем улучшить наше приложение!



Задания

1



Добавить новый товар

Создайте ещё один объект в массиве `products`.

2



Добавить поле `description`

Расширьте тип `Product` и mock-данные.

3



Вывести `description` в карточке

Отобразите описание товара в компоненте `ProductCard`.

4



Изменить текст кнопки

Придумайте и установите свой текст для кнопки.

5



Передать пустой массив

В `App.tsx` передайте `[]` вместо `products` и проверьте сообщение «No products found».



Совет

Не бойтесь экспериментировать! Пробуйте, проверяйте, делайте выводы.

Что должно получиться (пример)

Hide products



Apple iPhone 15

Category: Smartphones

Price: \$999

In stock

Latest iPhone with A17 Pro chip, amazing camera and all-day battery life.



Sony WH-1000XM5

Category: Headphones

Price: \$349

In stock

Industry-leading noise canceling headphones with exceptional sound quality.



Apple MacBook Air M2

Category: Laptops

Price: \$1199

In stock

Supercharged by the M2 chip, Lightweight design and all-day battery life.



Пример сообщения при пустом массиве:
No products found

Пример mock-данных (фрагмент)

```
1 export type Product = {
2   id: number;
3   title: string;
4   category: string;
5   price: number;
6   inStock: boolean;
7   description: string; // новое поле
8 };
9 export const products: Product[] = [
10   // ...
11 ];
```



Цель практики

Закрепить работу с:

- ✓ mock-данными
- ✓ props и компонентами
- ✓ списками и key
- ✓ условной отрисовкой
- ✓ состоянием и кнопками

Убедимся, что проект готов к использованию.



Что проверяем

- ✓ Код соответствует стандартам (**lint**).
- ✓ Сборка проекта проходит без ошибок.
- ✓ Приложение корректно отображается.
- ✓ Список товаров отображается и скрывается.
- ✓ Пустой список показывает сообщение **No products found**.
- ✓ Карточка товара содержит все поля (title, category, price, inStock, description).
- ✓ Ключи (**key**) заданы у элементов списка.



Полезные команды

```
npm run dev
```

— запуск сервера разработки

```
npm run build
```

— сборка проекта для продакшена



Критерии горности



Lint без ошибок



Сборка успешна



Интерфейс работает



Все условия задания выполнены



Проверяем в терминале

1 `npm run lint`

```
> eslint .  
✓ No problems found.  
✨ Done in 1.23s.
```



Успех

Код соответствует правилам ESLint.

2 `npm run build`

```
> tsc && vite build  
vite v5.1.0 building for production...  
✓ 142 modules transformed.  
dist/index.html                0.46 kB    gzip: 0.28 kB  
dist/assets/index-abc123.js    45.12 kB    gzip: 14.01 kB  
✓ built in 1.87s
```



Успех

Сборка прошла без ошибок.



Готово!

Поздравляем! Вы применили `proor` `pnpm`, списки и условную отрисовку.

Приложение работает корректно и готово к дальнейшему развитию. 🚀